CampusCloud Frontend — Architecture Diagram

Team A06 · BDS-8A

This document covers the internal architecture of the frontend dashboard only. Other teams' backend services and data-plane components are referenced only where the frontend directly interfaces with them.

High-Level Component Tree

```
<App>  (React Router)
│
├── /login       → <Login />
├── /register    → <Register />
│
└── <ProtectedRoute>        (redirects to /login if no token)
      └── <Layout>          (sidebar shell)
            ├── /dashboard  → <Dashboard />
            ├── /projects   → <Projects />
            └── /containers → <Containers />
```

Internal Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                     BROWSER  (React SPA)                             │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │                     AuthContext                          │       │
│   │  token ──────────────────────────────────────────────┐   │       │
│   │  user  ──────────────────────────────────────────┐   │   │       │
│   │  loginCtx() / logoutCtx()                        │   │   │       │
│   └──────────────────────────────────────────────────┼───┼───┘       │
│                                                      │   │           │
│   ┌──────────┐  ┌──────────┐  ┌──────────────┐   ┌──▼───▼──┐        │
│   │ Login /  │  │Dashboard │  │  Projects    │   │ Containers │      │
│   │ Register │  │          │  │              │   │          │        │
│   │  Pages   │  │stat cards│  │ list + create │  │deploy/del │       │
│   └──────────┘  └──────────┘  └──────────────┘   └──────────┘        │
│       │            │              │                    │             │
│   ────┴────────────┴──────────────┴────────────────────┴─────────────│
│                                                                      │
```

```
│                        API Layer  (src/api/)                    │
│                                                                 │
│     ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐    │
│     │   auth.js    │    │ projects.js  │    │  instances.js    │    │
│     │              │    │              │    │  (localStorage   │    │
│     │ login()      │    │ getProjects()│    │   mock)          │    │
│     │ register()   │    │ createProject│    │                  │    │
│     │ logout()     │    │ ()           │    │ getInstances()   │    │
│     └──────────────┘    └──────────────┘    │ createInstance() │    │
│            │                  │             │ deleteInstance() │    │
│            │                  │             │ getStats()       │    │
│            │                  │             └──────────────────┘    │
│            │                  │                     │               │
└────────────┼──────────────────┼─────────────────────┼───────────────┘
             │  Bearer JWT      │  Bearer JWT         │  localStorage
             │                  │                     │  (no network)
             ▼                  ▼                     ▼

    ┌──────────────────────────────────────┐     ┌──────────────────┐
    │   Backend REST API  (:5000)          │     │  localStorage    │
    │   (A01–A04, live)                    │     │  (mock data store)│
    │                                      │     │                  │
    │   POST /login                        │     │  "instances" key │
    │   POST /register                     │     │  JSON array      │
    │   POST /logout                       │     └──────────────────┘
    │   GET  /project                      │
    │   POST /project                      │
    └──────────────────────────────────────┘
```

Component Descriptions

Pages

Page	File	Responsibility
Login	`pages/Login.jsx`	Email + password form; calls `auth.js → login()` ; stores JWT via `AuthContext`
Register	`pages/Register.jsx`	Account creation; client-side validation (email domain, password strength); calls `auth.js → register()`
Dashboard	`pages/Dashboard.jsx`	Stat cards (projects, containers, CPU, memory); recent-projects list; recent-containers list
Projects	`pages/Projects.jsx`	Fetches and lists user projects; "New Project" modal calls `projects.js → createProject()`
Containers	`pages/Containers.jsx`	Lists containers (filtered by `?project=` param); "Deploy" modal; delete per card

		Deploy modal, delete per card

Shared Components

Component	File	Responsibility
Layout	`components/Layout.jsx`	App shell — renders `<Sidebar>` + `<Outlet />`
Sidebar	`components/Sidebar.jsx`	Navigation links; highlights current route
Modal	`components/Modal.jsx`	Generic dialog; closes on Escape key or backdrop click
ProtectedRoute	`components/ProtectedRoute.jsx`	Reads `AuthContext`; redirects to `/login` if no token

Context

Context	File	What it holds
AuthContext	`context/AuthContext.jsx`	`token` (JWT string), `user` (id + email), `loginCtx()`, `logoutCtx()`; persists to `localStorage` on every change

API Modules

Module	File	Real or Mock
auth	`api/auth.js`	**Real** — `fetch()` to `/login`, `/register`, `/logout`
projects	`api/projects.js`	**Real** — `fetch()` to `GET /project`, `POST /project`
instances	`api/instances.js`	**Mock** — reads/writes `localStorage`; no network calls yet

---

Data Flows

Login

```
User submits form
      │
      ▼
Login.jsx  →  auth.js → POST /login { email, password }
      │
```

```
      │
      ▼
Response: { accessToken, user }
      │
      ▼
AuthContext.loginCtx(token, user)
  ├── state updated
  └── localStorage updated
      │
      ▼
React Router → navigate("/dashboard")
```

Create Project

```
User clicks "New Project" → Modal opens
      │
      ▼
Projects.jsx submits name
      │
      ▼
projects.js → POST /project  { name }
  Authorization: Bearer <token>  (from AuthContext)
      │
      ▼
Response: { id, name, owner_id, created_at }
      │
      ▼
Projects.jsx appends to local state → list re-renders
```

Deploy Container (mock)

```
User clicks "Deploy" → Modal opens
      │
      ▼
Containers.jsx submits { name, image, project_id }
      │
      ▼
instances.js → createInstance()
  ├── generates a UUID
  ├── builds container object { id, name, image, project_id, status: "running", ..
  └── writes to localStorage["instances"]
      │
      ▼
Containers.jsx appends to local state → list re-renders
```

Delete Container (mock)

```
User clicks "Delete" on a container card
      │
      ▼
Containers.jsx calls deleteInstance(id)
      │
      ▼
instances.js → filters out the matching entry from localStorage["instances"]
      │
      ▼
Containers.jsx removes item from local state → list re-renders
```

State Management

There is **no external state library** (no Redux, no Zustand). State lives in two places:

Location	What lives there
`AuthContext` (React Context)	JWT token, logged-in user; shared across the entire app
Page-level `useState`	Lists (projects, containers), loading flags, modal open/close state

This keeps the footprint small and easy to reason about for a team codebase.

Routing Summary

Path	Component	Protected
`/login`	`Login`	No
`/register`	`Register`	No
`/dashboard`	`Dashboard`	Yes
`/projects`	`Projects`	Yes
`/containers`	`Containers`	Yes
`*`	Redirect → `/login`	—

Mock → Real API Swap Points

When the BDS-8B data plane is ready, **only `src/api/instances.js` needs to change**

When the BDC-OD data plane is ready, **only `src/api/instances.js` needs to change.** Replace each function body with a `fetch()` call to the real endpoint. No page components need to be modified.

Function	Planned real endpoint
`getInstances(projectId?)`	`GET /instances?project_id=<id>`
`createInstance(data)`	`POST /instance`
`deleteInstance(id)`	`DELETE /instance/:id`
`getStats()`	derived from `GET /instances`